MAZEN BAHGAT

DevOps | MLOps | AI-Native Software | Embedded Software | Model-Based Design

mazen.bahgat@outlook.com | +201121995449 | github.com/Mazen-Bahgat | Cairo, Egypt

AWS Certified Cloud Practitioner (Oct 2026)

PROJECT PORTFOLIO

18 projects across professional internship work, DevOps, MLOps, AI-native software engineering, cloud computing, embedded systems, control theory, robotics, and IoT.

PART A — PROFESSIONAL INTERNSHIP (RIVORA LABS, 2026)

Professional internship work delivering a production agricultural technology platform, spanning mechanical design, full-stack software engineering, and applied machine learning.

1. Agricultural AI, IoT & Remote Sensing Platform Project

Role: Internship | Rivora Labs, Egypt | Aug – Sep 2026

Category: Full-Stack Development / Geospatial Data Engineering / Applied ML / Mechanical Design

Deployment: Center-pivot irrigation structure (physical deployment, reclaimed agricultural land, Egypt)

Tools & Stack: React 18, TypeScript, Vite, TanStack Query, Vitest, Python 3.12, FastAPI, Uvicorn, Pydantic v2, Argon2, pyproj, NumPy, tifffile, SQLite, Autodesk Inventor, Ollama, OpenWebUI, Sentinel-2/Copernicus Data Space Ecosystem, Pirate Weather API, PyTorch

An agricultural technology platform combining mechanical pivot-irrigation design, satellite-driven crop and land health monitoring, and applied machine learning for disease detection on reclaimed land in Egypt. Ownership progressed through three distinct phases as the project evolved.

- Phase 1 — Mechanical Design: designed the full center-pivot irrigation structure in Autodesk Inventor, establishing the complete pivot geometry, and evaluated multiple linear rail system configurations to arrive at a mechanically sound, field-implementable design for real-world deployment.
- Phase 2 — Dashboard Development: built the project's monitoring dashboard end-to-end. Frontend in React 18 + TypeScript (Vite, TanStack Query, Vitest/Testing Library); backend in Python 3.12 with FastAPI + Uvicorn, Pydantic v2 for validation, Argon2 for authentication, pyproj for geospatial geometry, NumPy + tifffile for satellite raster processing, and SQLite for storage.
- Satellite data pipeline: integrated the Copernicus Data Space Ecosystem (CDSE), pulling Sentinel-2 L2A imagery via its Process/Catalog APIs at 10m resolution per field. Extracted spectral bands B04/B08 (NDVI), B05 red-edge (NDRE), and B11 SWIR (NDMI), with SCL/dataMask cloud-shadow masking — producing accurate crop and land health heatmaps.
- Weather data pipeline: integrated the Pirate Weather API (point + timemachine/forecast endpoints) for hourly humidity, temperature, precipitation, wind, and pressure — feeding a Cercospora disease-risk model.
- Local AI assistant integration: deployed a locally run qwen3.5:9b model served via OpenWebUI through Ollama. Architected the backend as a secure proxy layer (FastAPI to OpenWebUI's API, not iframed) so the frontend never exposes the model's URL, credentials, or system-policy controls — read-only/advisory, with streaming chat and image analysis (JPEG/PNG/WebP, EXIF-stripped) support, persisted per conversation.
- Phase 3 — Disease Classification & Detection: researched existing sugar-beet disease detection literature (Faster R-CNN, YOLOv4, ResNet50, VGG16/19, InceptionV3; reported accuracies 95–98% on controlled datasets) to inform architecture selection, then defined a PyTorch-based transfer-learning pipeline

(EfficientNet/MobileNet/ResNet baselines, with planned Vision Transformer comparison) for the project's specific crop and regional conditions.

PART B — UNIVERSITY, DEVOPS & AI/CLOUD PROJECTS (QUEEN'S UNIVERSITY, 2026)

Projects completed as part of the MSc Computing programme at Queen's University, demonstrating applied skills in AI-native software engineering, DevOps, CI/CD, GitOps, MLOps, and cloud infrastructure.

1. HerHealthGPT — Multilingual Women's Health Triage Assistant (MSc Capstone)

Role: MLOps/Systems Lead, 4-person team | Master's Project, Queen's University | Oct 2026

Category: Generative AI / RAG / Health AI Safety / MLOps / Cloud Deployment

Infrastructure: AWS EC2 (t3.large) + RDS PostgreSQL 16 | Campus GPU server (4 × NVIDIA A16) via Cloudflare tunnels

Tools & Stack: Qwen3.5-9B, QLoRA (Unsloth, PEFT), vLLM, Qwen3-Embedding-4B, LangChain, FAISS, Python 3.12, FastAPI, SQLAlchemy, Alembic, Pydantic, PostgreSQL 16, Next.js 16, React 19, TypeScript, Tailwind CSS, Docker Compose, Caddy, Terraform, GitHub Actions, AWS (EC2, RDS, ECR, S3, SES, Route 53, CloudWatch, SNS, Secrets Manager, SSM), Cloudflare Tunnel

Live: herhealthgpt.org | **GitHub:** github.com/HerHealthGPT/herhealthgpt | **Paper:** arxiv.org/abs/2609.20684

A live, safety-first triage companion that tells a woman how urgently to seek care for menstrual, PCOS and fertility concerns, and why, in English, French or Arabic, citing its sources and never diagnosing. Supervised by Dr. Rahatara Ferdousi, with industry mentors from the Deloitte Innovation Hub, Egypt.

- Benchmark first: co-authored HerHealthEval (published on arXiv), 540 matched items per language (90 cases × 6 writing styles) in English, French and Arabic, labelled for concern, risk, action and need for clarification. It showed headline accuracy hiding a safety failure: the base model beat the 64.4% baseline at 66.7% yet sent 71.8% of see-a-doctor cases to routine care.
- Model adaptation: compared five Qwen3.5-9B configurations, fine-tuned with QLoRA (LoRA rank 16, Unsloth + PEFT) on 2,700 question-answer pairs filtered from 136,109 candidates with a two-key leakage check. Fine-tuning cut clarification recall from 85.6% to 4.4%, and an audit traced a 99.4% French/Arabic under-triage collapse to English keyword labels; recomputing them reversed it.
- Guardrailed design: the model decides only within closed vocabularies through a Pydantic TriageDecision schema (ask, conclude, inform or decline), checked by six code gates and citation provenance checks, so a diagnosis cannot be represented. The red-flag gate lifted warning-sign escalation to 17 of 17 rules.
- Knowledge & retrieval: a source-traceable knowledge base of 39 features, 19 conditions, 150 quoted evidence rows and 17 warning-sign rules from NHS, CDC, MedlinePlus, NICHD and OWH; retrieval over 3,862 source sentences with LangChain, FAISS and Qwen3-Embedding-4B. Relevant retrieval raised concern accuracy by 5.19 points (95% CI 1.11 to 9.26).
- Model serving: three vLLM servers on 4 × NVIDIA A16 GPUs (Qwen3.5-9B across two GPUs with the LoRA adapter in the same process, embeddings on a third) at temperature 0 for repeatable replies; fixed a vLLM all-reduce fault that silently corrupted activations. The GPU server is reached over Cloudflare tunnels with a watchdog that reopens a dead tunnel every 5 minutes.
- Cloud deployment: Next.js and FastAPI containers (Docker Compose) on EC2 behind Caddy with Let's Encrypt; RDS PostgreSQL 16 in a private subnet with 7-day backups; Route 53, SES, Secrets Manager and SSM, with every AWS resource in Terraform. CloudWatch logs and 7 alarms (server, CPU, memory, disk, database) route to SNS, plus a monthly budget alert.
- CI/CD: a merge to main builds both images, pushes Git SHA-tagged images to ECR and deploys through AWS Systems Manager; the server migrates the database, health-checks the release through Caddy and rolls back automatically on failure. Path-filtered workflows test each service, plan Terraform and block files that look like secrets.
- Outcome: live at herhealthgpt.org with accounts, chat history, cycle-data import and right-to-left Arabic; 25 of 30 user-study participants preferred the full system over plain-model chat. 20 architecture decision records document every design change.

2. TestSeed — AI-Powered MongoDB Seed Data Generator

Course: CISC818: Software Engineering with AI | Queen's University | Jun 2026

Category: AI-Native Software Engineering / Full-Stack / Spec-Driven Development

Infrastructure: Cloud / Local development

Tools & Stack: Spec Kit, ChatGPT, Cursor with Codex, OpenAI API, TypeScript, Next.js, MongoDB, shadcn/ui, MCP (Model Context Protocol) — Context7, shadcn, MongoDB, next-devtools

A developer tool that generates realistic, schema-aware MongoDB seed data in dependency order using AI, with validation, preview, and inline-editing built around the model's output rather than trusting it directly. Built individually as part of a 4-person team project.

- Owned three connected features — AI Seed Generation, Record-Count Controls, and Preview/Inline Editing — using a spec-driven development workflow with Spec Kit, ChatGPT, Cursor with Codex, and the OpenAI API.
- Used Spec Kit (/speckit-specify, /speckit-plan) to convert acceptance criteria into a structured specification and plan before any implementation code existed, then /speckit-tasks broke the AI Seed Generation epic into 63 dependency-ordered tasks across 5 user stories.
- Ran /speckit-implement to complete all 63 tasks end-to-end in 12 minutes 21 seconds, producing shared generation contracts, core generation modules (dependency planning, validation, retry orchestration, chat refinement), new API routes, and a Generate page with live JSON preview.
- Designed the AI generation pipeline to guarantee parent-before-child ObjectId reference integrity: the model is prompted to generate parent collections first and use their generated IDs as context for child collections, with post-generation validation and a retry prompt that feeds back any violated constraint.
- Built record-count validation that treats count mismatches (e.g. requesting 3 records and receiving 0 or 5) as hard validation errors rather than silently accepted model variation.
- Built the preview and inline-editing UI (with shadcn MCP scaffolding and custom commit-on-blur/Enter-key logic): edited records are revalidated server-side, and export/seeding is blocked on any type, enum, required-field, uniqueness, or reference violation — with unsafe fields (IDs, references, nested objects) kept read-only.
- Debugged real AI generation failures — missing required fields, count mismatches, invalid ObjectId types — by converting observed failures into stronger prompt constraints, better default values, and a tightened validator, reinforcing a systems-level view: 'AI is powerful for generating candidates, but software engineering is responsible for enforcing correctness.'

3. CI/CD Pipeline, GitOps Deployment & Observability Platform

Course: CISC814: DevOps and Continuous Delivery | Queen's University | Summer 2026

Category: DevOps / CI/CD / GitOps / Observability

Infrastructure: VM (Ubuntu, localuser@130.15.5.209) | k3d local Kubernetes cluster

Tools & Stack: Gitea, Gitea Actions, Python, FastAPI, Ruff, Mypy, pytest, Docker, Syft (SBOM/SPDX), Kubernetes, ArgoCD, Kustomize, Argo Rollouts, Prometheus, Grafana, k3d

GitHub: https://github.com/Mazen-Bahgat/ml-sentiment-cicd.git | https://github.com/Mazen-Bahgat/devops-sentiment-k8s-gitops.git

Demo: drive.google.com/file/d/1JrWXD9OctY18s7I3XvVHQgY5bruHw9RI/view?usp=sharing

An end-to-end DevOps platform for a FastAPI ML sentiment analysis service — covering automated CI, containerisation, GitOps deployment, progressive canary releases, and full observability.

- Part A — CI Pipeline: 7-step Gitea Actions pipeline triggered on push/PR/tag: (1) dependency install + health check, (2) Ruff linting, (3) Mypy static type checking with disallow_untyped_defs=true, (4) pytest with 100% code coverage across all 3 modules (14/14 tests, threshold 70%), (5) Docker multi-stage build using python:3.11-slim with non-root appuser, (6) image push tagged with :latest and full Git SHA, (7) Syft SBOM in SPDX JSON — all reports stored as CI artefacts.
- Configured branch protection on main requiring a passing CI run before any merge, enforcing quality gates across the full team.
- Part B — GitOps: Kustomize base/overlay structure — base defines Rollout, ConfigMap, ClusterIP services; production overlay patches replicas, CPU/memory limits, and pins image to deployed Git SHA. ArgoCD deployed with selfHeal: true and prune: true, reconciling every ~30 seconds.

- Part B — Canary Deployment: 9-step Argo Rollout shifting traffic 20%→40%→60%→80%→100% with 30s pauses; stable and canary pods served via separate ClusterIP services.
- Part B — Observability: FastAPI instrumented with 3 Prometheus metrics (requests counter, label counter, latency histogram); ServiceMonitor scrapes /metrics every 15s. Grafana dashboard (provisioned via ConfigMap) with 4 live panels: request rate, HTTP 5xx error rate, latency p50/p95/p99, and CPU/memory per pod.
- Full flow: code push → CI → Docker push → GitOps repo update → ArgoCD sync → Argo Rollout canary → Prometheus scrape → Grafana live metrics. Recorded 10-minute live demo.

4. MLOps Pipeline: Experimentation, Serving, A/B Testing & Drift Monitoring

Course: CISC814: DevOps and Continuous Delivery | Queen's University | Summer 2026

Category: MLOps / LLM Experimentation / A/B Testing / Drift Monitoring / Canary Deployment

Infrastructure: VM (Ubuntu, localuser@130.15.5.209) | k3d cluster2 | GPU (nvidia.com/gpu=1)

Tools & Stack: Python, MLflow, vLLM (v0.18.0), Qwen2.5-1.5B, GPTQ (GPTQModel, Marlin kernel), FastAPI, Evidently AI, Prometheus, Pushgateway, Grafana, ArgoCD, Argo Rollouts, Kubernetes, k3d, Docker

GitHub: github.com/Mazen-Bahgat/sentiment-mlops-pipeline.git

Demo: drive.google.com/file/d/1_ymkmRLPZlDzRARHpfCn_E3_yGU_Eumr/view?usp=sharing

A production-grade MLOps system evolving a TextBlob sentiment service to Qwen2.5-1.5B — covering MLflow experimentation, vLLM model serving, A/B traffic routing, Evidently drift monitoring, and drift-gated canary promotion in Kubernetes.

- Part A — MLflow Experimentation: Configured MLflow tracking server (http://localhost:5050) with artifact proxying (mlflow-artifacts:/). Ran 3 few-shot experiments (1/3/5-shot) on Qwen2.5-1.5B using vLLM, logging params, metrics (accuracy, F1, latency), and artefacts to nested MLflow runs. Key result: 3-shot achieves F1 = 1.000; 1-shot scores F1 = 0.896; 5-shot adds no further gain.
- Part A — GPTQ Quantization: Quantized Qwen2.5-1.5B to 4-bit GPTQ (bits=4, group_size=128, wikitext-2 calibration, Marlin kernel via nvcc). Results: 3.6x latency improvement (0.327s → 0.092s), 65% model size reduction (3.09 GB → 1.08 GB), and only 10% accuracy drop — a favourable production trade-off.
- Part A — Model Registry: registration.py queries MLflow for the best F1 run per experiment type, creates a versioned model under sentiment-qwen2.5, assigns champion (V1 fp16) and challenger (V2 GPTQ) aliases, and tags each version with vllm_model_path for Part B retrieval.
- Part B — vLLM Serving: Retrieved model paths from MLflow aliases via get_model_version_by_alias(); started V1 (port 8001, 2.91 GiB, FlashAttention v2) and V2 (port 8002, 1.12 GiB, gptq_marlin kernel) simultaneously — total GPU usage 4.03 GiB on an A16 (15.36 GiB VRAM).
- Part B — A/B Traffic Router: FastAPI router (port 8000) with 80/20 V1/V2 weighted random routing. Served 250 predictions — 193 to V1 (77.2%) and 57 to V2 (22.8%), matching the 80/20 target.
- Part B — Drift Monitoring: Evidently DataDriftPreset compared V1 (193 reference samples) vs V2 (57 current samples) — drift share = 0.0, no distributional shift — GPTQ quantization preserved decision boundaries.
- Part B — Kubernetes Canary: Deployed to k3d (cluster2) with GPU node + volume mounts. Argo Rollouts canary with drift-gated AnalysisRun: sentiment-analysis-job fires at each step, computes drift_score, pushes to Pushgateway, Prometheus queries sentiment_drift_score < 0.5 — 3 consecutive passes → weight progresses 20%→50%→80%→100%. V2 fully auto-promoted; V1 scaled down.
- Grafana MLOps Drift Analysis dashboard provisioned via ConfigMap: drift score timeline, gauge (threshold 0.5), and Pushgateway metrics table — drift score held flat at 0 throughout the canary promotion.

5. Cyber SOC Chatbot — Cloud-Based AI Security Triage Assistant

Course: CISC886: Cloud Computing | Queen's University | May 2026

Category: Cloud Computing / MLOps / DevSecOps / LLM Fine-Tuning

Infrastructure: AWS EC2 (M5.xlarge), AWS EMR (m5.xlarge Spark cluster), On-Prem GPU (RTX 5000 Ada, 16 GB VRAM)

Tools & Stack: AWS (VPC, EC2, S3, EMR, IAM, EIP, Security Groups), Terraform (IaC), PySpark,

Python, TinyLlama-1.1B, QLoRA/PEFT, Unsloth, HuggingFace Transformers, bitsandbytes, Ollama, GGUF, OpenWebUI, Docker, Bash

GitHub: github.com/Mazen-Bahgat/Cyber-SOC-Chatbot

An end-to-end cloud-native pipeline to fine-tune, deploy, and serve a cybersecurity SOC triage assistant on AWS — covering infrastructure provisioning, big data preprocessing, LLM fine-tuning, GGUF export, and REST API serving.

- Provisioned full AWS infrastructure using Terraform (IaC): custom VPC (10.0.0.0/16), public and private subnets, Internet Gateway, NAT Gateway, EC2 security groups (SSH, HTTP, HTTPS), IAM roles, S3 buckets, and Elastic IP — all reproducible from a single terraform apply.
- Preprocessed the Witfoo Precinct6 Cybersecurity Dataset (114M+ records from HuggingFace) using a PySpark pipeline on AWS EMR (EMR 6.x, Spark 3.5, m5.xlarge × 3 nodes), performing undersampling to balance 3 classes to 150K total.
- Fine-tuned TinyLlama-1.1B-Chat-v1.0 using QLoRA (r=16, 4-bit NF4) via Unsloth + TRL SFTTrainer — training loss reduced from 0.5518 to 0.2255 (−58%), validation loss 0.1658 with no overfitting — completed in 41 minutes on a single RTX 5000 Ada GPU.
- Exported LoRA adapter to GGUF, uploaded to S3, deployed on EC2 via Ollama REST API (port 11434), served through Dockerised OpenWebUI (port 3000) auto-starting via systemd.
- DevSecOps best practices: least-privilege IAM roles, private subnet isolation, SSH restricted to single IP, encrypted EBS storage.
- Fine-tuned model produces structured SOC triage assessments (attack classification, risk level, recommended action, IOC extraction) vs base model's generic responses.

6. Guarded RAG Tutor — Safe Retrieval-Augmented Generation System

Course: CSAI810: Topics in AI — Generative AI Fundamentals | Queen's University | 2026

Category: Generative AI / RAG / Safety **Infrastructure:** Cloud / Local

Tools & Stack: Python, Gemini 2.5 Flash, ChromaDB, OpenAI Moderation API, Sentence Transformers (all-MiniLM-L6-v2), LangChain, PyMuPDF, Gradio

GitHub: github.com/Mazen-Bahgat/Guarded-RAG-Information-Assistant

A safe, document-grounded question-answering system combining RAG with multi-layer content guardrails to ensure responses are accurate, context-based, and policy-compliant.

- Designed a full RAG pipeline: PDF/text documents chunked via LangChain Text Splitter, embedded with Sentence Transformers (all-MiniLM-L6-v2), stored in ChromaDB in-memory vector database.
- Scope classification layer (Gemini 2.5 Flash) rejects off-topic or non-educational queries before generation; OpenAI omni-moderation-latest guardrail applied on both input and output in real time.
- Gemini 2.5 Flash used as worker model, grounding all responses strictly in retrieved document context.
- Deployed via Gradio for live document upload and real-time Q&A; full source code and demo video published on GitHub.
- Demonstrated end-to-end responsible AI: scope filtering → retrieval → generation → post-generation safety moderation.

PART C — ENGINEERING PROJECTS (MECHATRONICS & EMBEDDED SYSTEMS)

Projects completed during the BEng Mechatronics programme at Arab Academy for Science and Technology and through independent engineering work.

1. ADAS with Firmware Over-the-Air (FOTA) Update System

Category: Automotive / Embedded Systems / Wireless **Hardware:** STM32 Bluepill, ESP8266, AI Thinker A9G GSM Module

Tools: Embedded C, UART/SPI, GPS, GSM, FOTA OTA Framework

A fully integrated ADAS prototype for an autonomous model car featuring safety functions and over-the-air firmware updates.

- Implemented Adaptive Cruise Control using ultrasonic sensors to maintain a safe following distance automatically.
- Developed an Auto Parking algorithm that detects a suitable space and executes a parallel parking manoeuvre without driver input.
- Built a Lane Tracking system using IR sensors to keep the vehicle within lane boundaries at all times.
- Integrated GPS tracking via A9G module for real-time vehicle location monitoring from a remote dashboard.
- Designed FOTA update mechanism using ESP8266 Wi-Fi, enabling remote firmware flashing without physical hardware access.
- Structured firmware using a modular, layered architecture for maintainability and extensibility across ECUs.

2. IoT Inverted Pendulum Control Experiment

Category: Control Systems / IoT / HIL **Hardware:** ATmega32, ESP8266

Tools: MATLAB, Simulink, Simscape, IoT Application

An IoT-based rotary inverted pendulum stabilised using a Linear Quadratic Regulator (LQR) with swing-up functionality.

- Implemented an LQR controller for balancing the rotary inverted pendulum, a classic nonlinear control challenge.
- Developed a swing-up algorithm to transition the pendulum from the stable downward position to the unstable upright before engaging LQR.
- Created a state-space model in MATLAB to capture nonlinear dynamics and design the optimal LQR gain matrix.
- Built a Hardware-in-the-Loop (HIL) model in MATLAB Simulink for full offline testing before hardware deployment.
- Designed an IoT mobile application to remotely adjust LQR gains, monitor states in real time, and stream live video.
- Achieved stable balancing on physical hardware with fully wireless operation and real-time parameter tuning.

3. IoT Linear Servo Base Unit Control Experiment

Category: IoT / Control Systems / Embedded **Hardware:** ATmega32, ESP8266, ESP32-CAM

Tools: MATLAB, Simulink, Embedded Coder, Custom IoT Application

A remotely monitored and controlled gantry plate using PID control, IoT connectivity, and auto-generated embedded code.

- Designed and implemented a PID-controlled gantry plate with real-time remote monitoring over Wi-Fi.
- Used MATLAB Embedded Coder to auto-generate production-ready C code for the PID controller.
- Integrated ESP32-CAM for live video streaming, enabling remote visual monitoring of the plant.
- Built a multi-widget IoT application for parameter tuning, setpoint adjustment, and live telemetry display.

4. 6-Degree of Freedom Robotic Arm Model

Category: Robotics / Model-Based Design / Real-Time Control **Hardware:** Arduino Mega

Tools: MATLAB, Simulink, Simscape, Autodesk Inventor

Full design-to-deployment pipeline of a 6-DOF serial robotic arm from CAD modelling to real-time PID control.

- Modelled full robot geometry in Autodesk Inventor and exported to MATLAB Simscape for physics-based simulation.
- Developed forward and inverse kinematics solvers and applied PID control on each joint axis.
- Achieved real-time control via Simulink communicating joint commands to an Arduino Mega over serial.
- Validated trajectory tracking through simulation before deploying on the physical prototype.

5. Fire Fighting Robot

Category: Robotics / Competition **Hardware:** ATmega32

Tools: Embedded C, Flame Sensors, DC Motor Control

An autonomous fire detection and extinguishing robot. First Place — ST Smart Competition, Egyptian Engineers Syndicate.

- Equipped with multi-directional flame sensors for 360-degree fire source detection.
- Programmed autonomous navigation to approach the flame source while avoiding obstacles.
- Activated a water pump when within range to extinguish the simulated fire automatically.
- Awarded First Place at the ST Smart Competition organised by the Egyptian Engineers Syndicate.

6. Radar Camera Security System

Category: Embedded Systems / IoT / Security **Hardware:** ATmega32, SIM808 GSM/GPS Module, ESP32-CAM

Tools: Embedded C, Sensor Fusion, SMS Alerts, Video Surveillance

A multi-sensor security system protecting radar installations against theft and environmental threats.

- Integrated vibration, water, and temperature sensors for physical and environmental threat detection.
- Employed ESP32-CAM for continuous video surveillance triggered by threat events.
- Sent automated SMS and GPS location alerts via SIM808 upon intrusion detection.
- Demonstrated multi-sensor fusion and real-time embedded event handling in a security context.

7. Smart Home Automation System

Category: IoT / Embedded Systems **Hardware:** ESP8266 (NodeMCU)

Tools: Embedded C, Mobile Application, Wi-Fi Control

A Wi-Fi-enabled smart home controller for managing appliances and security from a mobile application.

- Designed firmware to control lighting, fans, and appliances remotely via mobile app.
- Implemented security features including door lock control and motion detection alerts.
- Built a companion mobile application with toggle controls and real-time device status feedback.

8. Automated Warehouse Delivery Robot

Category: Robotics / Automation **Hardware:** Arduino Mega

Tools: LabVIEW, Serial Communication

An autonomous ground robot for warehouse delivery supervised through a LabVIEW graphical interface.

- Programmed motor control, obstacle avoidance, and line following on the Arduino Mega.
- Developed a LabVIEW supervisory interface for command dispatch and live status tracking.
- Integrated serial communication between the PC host and embedded controller for real-time exchange.

9. Conveyor Belt PID Speed Control

Category: Control Systems / Embedded Systems **Hardware:** ATmega328P

Tools: Embedded C, PID Algorithm, Encoder Feedback

A closed-loop speed control system maintaining precise conveyor velocity under varying load conditions.

- Implemented a discrete PID controller using encoder pulse counting for speed feedback.

- Tuned PID gains (Kp, Ki, Kd) to achieve stable speed regulation with steady-state error elimination.
- Applied PWM output to a motor driver for variable speed actuation.

10. Automated Mixing and Filling Machine

Category: Industrial Automation / Embedded Systems **Hardware:** PIC16F877A

Tools: Embedded C, Sensors, Actuators

A microcontroller-based machine automating ingredient mixing and precise container filling.

- Programmed the PIC16F877A to sequence mixing stages with configurable timing and ratios.
- Controlled stepper motors and solenoid valves for precise dispensing and filling.
- Integrated level sensors for automated stop/start logic and overflow protection.

11. Smart Office Automation System

Category: IoT / Embedded Systems **Hardware:** Arduino Mega

Tools: Embedded C, Sensors, Relay Modules

An integrated smart office system automating lighting, temperature control, and door management.

- Automated lighting using PIR sensors to switch lights on/off based on occupancy.
- Implemented temperature monitoring with automatic fan/AC relay control.
- Integrated door access control with servo-actuated lock and event-driven firmware architecture.